

DEADLOCK GUARDIAN

Deadlock Risk Analizi, Tespiti ve Kurtarma Sistemi

Proje Raporu

İşletim Sistemleri Dersi · Yazılım Mühendisliği Bölümü

Ana kaynak: Silberschatz, Galvin, Gagne — *Operating System Concepts*, 10. baskı,
Bölüm 8

Kaynak kod: github.com/eynalliomer/g.o.a.t-deadlock-guardian-project

Hazırlayanlar:

İçindekiler

1. Özet	3
2. Giriş ve Problem Tanımı	4
3. Teorik Arka Plan	5
4. Tasarım	6
5. Gerçekleştirme	8
6. Test ve Sonuçlar	10
7. Sonuç ve Gelecek Çalışmalar	15
8. Kaynakça	16
9. Ekip Katkıları	17

1. Özet

Deadlock Guardian, process'lerin kaynak kullanımını izleyen, deadlock riskini oluşmadan önce değerlendiren, oluşan deadlock'u tespit eden ve sistemi yeniden güvenli duruma getirmek için kurtarma seçenekleri sunan bir işletim sistemi **simülasyonudur**. Gerçek işletim sistemi process'leri yerine kontrollü bir modelde çalışır; senaryolar JSON dosyalarıyla tanımlanır.

Sistem üç aşamadan oluşur: (1) **risk analizi**: Banker's Algorithm ile güvenli durum kontrolü ve LOW / MEDIUM / HIGH / CRITICAL risk seviyeleri; (2) **tespit**: Resource Allocation Graph (RAG) üzerinde DFS ile döngü arama ve çok örnekli kaynaklar için tespit algoritması; (3) **kurtarma**: maliyete göre kurban seçerek process sonlandırma ya da kaynak geri alma.

Proje Python ile geliştirildi ve Flask tabanlı bir web arayüzüyle sunuldu. Arayüzde senaryolar adım adım oynatılabilir, RAG canlı çiziliyor, deadlock anında kurtarma seçenekleri maliyetleriyle listeleniyor ve tek tıkla uygulanabiliyor. Sistem **140 otomatik testle** doğrulandı; kod kapsamı **%99**'dur. Banker's Algorithm ders kitabındaki örnekle birebir aynı sonuçları vermektedir.

Ölçüt	Değer
Programlama dili / arayüz	Python 3.14 / Flask web arayüzü
Kaynak kod	18 modül, yaklaşık 1600 satır (src/)
Senaryo dosyası	10 (normal, tek ve çoklu deadlock, güvensiz durum, kurtarma, final demo)
Otomatik test / kod kapsamı	140 test, hepsi geçiyor / %99
Geliştirme süreci	11 haftalık plan, her hafta Git etiketiyle (hafta-01 ... hafta-11) işaretlendi

2. Giriş ve Problem Tanımı

Çok görevli bir işletim sisteminde process'ler sınırlı sayıdaki kaynağı (yazıcı, disk, kilit, bellek bölgesi) paylaşır. Bir process elindeki kaynağı bırakmadan başka bir kaynağı beklerken, o kaynak da ilk process'in kaynağını bekleyen başka bir process'in elindeyse, ikisi de sonsuza kadar bekler. Bu duruma **deadlock** (kilitlenme) denir.

Tanım

Deadlock: Bir process kümesindeki her process'in, yine aynı kümedeki başka bir process'in yol açabileceği bir olayı (genellikle kaynak bırakılmasını) beklemesi durumu (Silberschatz, 8.2).

Deadlock'un bedeli yüksektir: kilitlenen process'ler hiç ilerlemez, tuttukları kaynaklar başka process'lere de kapalı kalır ve sistem giderek yavaşlar. Gerçek sistemlerin çoğu deadlock'u görmezden gelir ve sorun çıktığında sistemin yeniden başlatılmasına güvenir; çünkü önleme ve kaçınma yöntemleri pahalıdır.

2.1 Projenin amacı

Deadlock Guardian'ın amacı, ders kitabında ayrı ayrı anlatılan deadlock kavramlarını **tek bir çalışan sistemde** birleştirmek ve deadlock'un oluşumunu, tespitini ve çözümünü görünür kılmaktır:

- Deadlock oluşmadan **önce** tehlikeyi fark edip kullanıcıyı uyarmak (risk analizi),
- Oluşan deadlock'u ve ona dahil process ile kaynakları **göstermek** (tespit),
- Deadlock'u en düşük maliyetle **çözmek** ve sistemin güvenli duruma döndüğünü doğrulamak (kurtarma),
- Bütün bunları adım adım izlenebilen, etkileşimli bir arayüzde sunmak.

3. Teorik Arka Plan

3.1 Deadlock'un dört gerekli koşulu (Coffman koşulları)

Bir sistemde deadlock oluşabilmesi için aşağıdaki dört koşulun **aynı anda** sağlanması gerekir (Silberschatz, 8.3.1). Koşullardan biri bile kırılırsa deadlock oluşmaz.

Koşul	Anlamı	Projedeki karşılığı
Karşılıklı dışlama (mutual exclusion)	Bir kaynak birimini aynı anda tek process kullanabilir	Resource.allocation: her birim tek process'te
Tut ve bekle (hold and wait)	Process kaynak tutarken başka kaynak bekleyebilir	Process elinde R1 varken R2'yi isteyip WAITING olabilir
Geri alınamazlık (no preemption)	Kaynak sahibinden zorla alınamaz	Kaynak yalnızca release ile bırakılır (kurtarma hariç)
Döngüsel bekleme (circular wait)	$P1 \rightarrow P2 \rightarrow \dots \rightarrow Pn \rightarrow P1$ bekleme zinciri	RAG'deki döngü ($P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$)

3.2 Resource Allocation Graph (RAG)

RAG, sistemin anlık durumunu gösteren yönlü bir graftır (Silberschatz, 8.3.2). Döğümlemler process'ler ve kaynaklardır. **İstek kenarı** $P \rightarrow R$, process'in kaynağı beklediğini; **atama kenarı** $R \rightarrow P$, kaynağın bir biriminin process'e verildiğini gösterir. Her kaynak tek örnekliyse grafta döngü olması deadlock olması demektir. Çok örnekli kaynaklarda ise döngü yalnızca **olası** bir deadlock'u gösterir: döngüdeki bir kaynağın başka bir birimi, kimseyi beklemeyen bir process'teyse o process bitince döngü çözülür.

3.3 Tespit algoritması

Çok örnekli kaynaklar için döngü aramak yetmediğinden ders kitabındaki tespit algoritması kullanılır (Silberschatz, 8.7.2). Algoritma, isteği mevcut boş kaynakla (*Work*) karşılanabilen bir process arar; bulursa o process'in işini bitirip kaynaklarını bırakacağını varsayar ($Work += Allocation$). Bu, yeni bir process bitemeyene kadar tekrarlanır; sonunda bitemeyen process'ler deadlock'tadır.

3.4 Banker's Algorithm ve güvenli durum

Banker's Algorithm (Dijkstra, 1965) deadlock'tan **kaçınma** yöntemidir (Silberschatz, 8.6.3). Her process baştan en fazla ne kadar kaynak isteyebileceğini (*Max*) bildirir. $Need = Max - Allocation$, process'in en kötü durumda daha isteyebileceği miktardır. Process'lerin her biri sırası geldiğinde *Need*'inin tamamını alıp bitirebileceği bir sıra (güvenli sıra) varsa sistem **güvenli durumdadır**. Bir istek, verildiğinde sistem güvenli kalacaksa verilir.

Tanım

Güvensiz durum $\neq$ deadlock. Deadlock olan her sistem güvensizdir; ama güvensiz bir sistemde deadlock henüz oluşmamış olabilir. Güvensiz durum, deadlock'un artık garanti olarak önlenemeyeceği anlamına gelir.

Tespit algoritması ile güvenlik algoritması aynı döngüyü kullanır; farkları talep tablosudur: tespit *Request*'e (şu an beklenen), Banker's *Need*'e (en kötü durumda istenebilecek) bakar.

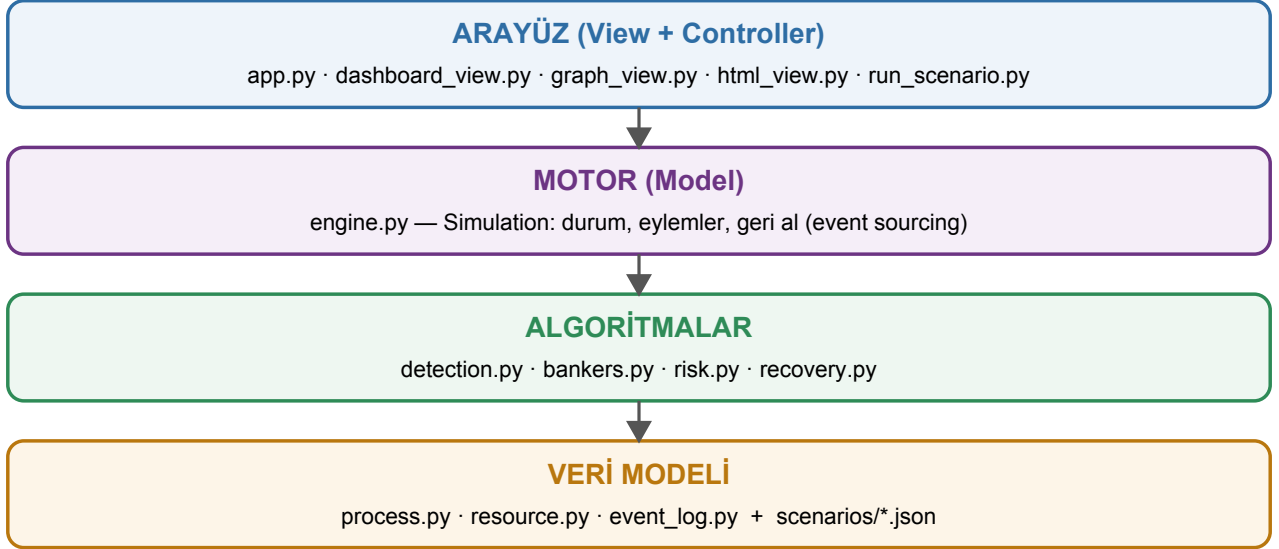
3.5 Kurtarma (recovery)

Deadlock oluştuğundan sonra döngü zorla kırılmalıdır (Silberschatz, 8.8). İki yöntem vardır: **process sonlandırma** (deadlock'taki process'ler birer birer sonlandırılır, her seferinde tespit tekrar çalıştırılır) ve **kaynak geri alma** (bir process'in kaynağı alınıp bekleyene verilir; kaynağı alınan process geri sarılır, rollback). Her iki yöntemde de amaç **en düşük maliyetli kurbanı** seçmek ve aynı process'in sürekli kurban seçilmesini (starvation) önlemektir.

4. Tasarım

4.1 Katmanlı mimari

Sistem, **sorumlulukların ayrılması** (separation of concerns) ilkesine göre dört katmana ayrıldı. Arayüz MVC (Model-View-Controller) yapısındadır: motor (Model) ekranı bilmez; aynı motoru hem komut satırı aracı hem web arayüzü kullanır. Algoritmalar arayüzden bağımsız olduğu için ayrı ayrı test edilebildi.



Her katman yalnızca altındakini çağırır; motor ekranı, algoritmalar motoru bilmez.

Şekil 1 — Deadlock Guardian'ın katmanlı mimarisi

4.2 Modüller

Katman	Modül	Sorumluluğu
Veri modeli	process.py	Process: durum (READY/WAITING/TERMINATED), tuttuğu kaynaklar, Max, öncelik
	resource.py	Resource: toplam/boş birim, dağıtım tablosu, bekleme kuyruğu, acquire/release
	event_log.py	Bütün kaynaklar için ortak, zaman sıralı olay kaydı
Algoritmalar	detection.py	RAG kurulumu, DFS ile döngü arama, tespit algoritması, deadlock raporu
	bankers.py	Need, güvenlik algoritması, kaynak isteği algoritması (SAFE/UNSAFE/WAIT)
	risk.py	LOW/MEDIUM/HIGH/CRITICAL risk seviyesi ve nedenleri
	recovery.py	Kurtarma seçenekleri, maliyet formülü, sonlandırma, kaynak geri alma
Motor	engine.py	Simulation: senaryoyu oynatma, elle eylem, analiz, geri al
Arayüz	app.py, dashboard_view.py	Flask web arayüzü (Controller ve View)
	graph_view.py, html_view.py	RAG'in SVG çizimi, kartlar, statik adım adım önizleme
	run_scenario.py	Komut satırı aracı: senaryoyu oynatır, konsol raporu ve HTML üretir

4.3 Veri yapıları

Bütün tablolar Python sözlükleriyle tutuldu; bu, ders kitabındaki matrislerin (Allocation, Max, Need) kaynak adlarıyla okunabilir bir karşılığıdır:

```
resource.allocation = {P1: 2, P3: 1}          # hangi process kaç birim tutuyor
resource.waiting_queue = [(P2, 1)]           # kim, kaç birim bekliyor
process.held_resources = {"R1": 2, "R2": 1}
process.max_claim    = {"R1": 2, "R2": 1}     # Banker's için bildirilen Max
rag                  = {"R1": ["P1"], "P2": ["R1"], ...} # {düğüm: [komşular]}
```

4.4 Senaryo dosyası biçimi

Senaryolar kod değiştirmeden yeni durumlar denenebilsin diye JSON dosyası olarak tanımlandı. *processes* bölümü isteğe bağlıdır; Max bildirilmezse Banker's analizi yapılmaz.

```
{
  "resources": [{"name": "R1", "total_instances": 1}, {"name": "R3", "total_instances": 2}],
  "processes": [{"name": "P1", "max": {"R1": 1, "R3": 1}, "priority": 1}],
  "events": [
    {"action": "acquire", "process": "P1", "resource": "R1"},
    {"action": "release", "process": "P1", "resource": "R1"},
    {"action": "recover"}
  ]
}
```

4.5 Tasarım kararları

Karar	Seçenek	Gerekçe
Kaynak isteme kuralı	Tüm-ya-da-hiç	İstenen miktarın tamamı verilemiyorsa process tamamen bekler; Banker's modeliyle uyumlu
Banker's davranışı	Yalnızca uyarı	Güvensiz istek engellenmez; kullanıcı riski görür ve deadlock'un nasıl oluştuğu izlenebilir
Risk: MEDIUM	Bekleme VEYA boş kaynak %20'nin altı	LOW → MEDIUM geçişi erken ve net görünsün
Kurtarma yöntemi	Birer birer sonlandırma + kaynak geri alma	Kitaptaki iki yöntem de gösterilsin; hepsini birden sonlandırmak gereksiz iş kaybı
Kurban maliyeti	öncelik×10 – serbest bırakılan birim×2 + kurban sayısı×5	Önemli process korunur, çok kaynak açan seçenek ucuzdur, sık kurban seçilen pahalılaşır (starvation)
Arayüz	Flask web	Hafta 5'ten beri yazılan HTML/SVG çizim kodu yeniden kullanıldı
Veri	JSON senaryo dosyaları	Okunabilir, sürüm kontrolüne uygun, ek kurulum gerektirmez

5. Gerçekleştirme

5.1 Döngü arama: DFS ve üç renk

Her düğüm beyaz (ziyaret edilmedi), gri (şu anki yolun üstünde) ya da siyah (bitti) olarak işaretlenir. Gri bir düğüme yeniden ulaşmak, yolun kendisine geri dönmesi, yani döngüdür. Siyah düğümler bir daha taranmadığı için algoritma $O(V + E)$ zamanda çalışır. Birden fazla bağımsız deadlock olduğunda bulunan döngünün düğümleri graftan çıkarılıp arama tekrarlanır (*find_cycles*).

```
def dfs(node):
    color[node] = GRAY
    path.append(node)
    for neighbor in graph[node]:
        if color[neighbor] == GRAY:           # yoldaki düğüme geri döndük: döngü
            return path[path.index(neighbor):]
        if color[neighbor] == WHITE:
            cycle = dfs(neighbor)
            if cycle:
                return cycle
    color[node] = BLACK
    path.pop()
```

5.2 Tespit ve Banker's için ortak çekirdek

Tespit algoritması ile Banker's güvenlik algoritması aynı Work/Finish döngüsünü kullandığı için döngü tek bir fonksiyona çıkarıldı; iki algoritma yalnızca farklı bir talep tablosu verir.

```
def run_to_completion(work, demand, allocation, finished):
    # demand: tespitite Request, Banker's'ta Need
    while progress:
        for name in finished:
            if not finished[name] and all(n <= work[r] for r, n in demand[name].items()):
                for r, n in allocation[name].items():
                    work[r] += n                # işini bitirir, kaynaklarını bırakır
                finished[name] = True
                order.append(name)
    return order, stuck                        # güvenli sıra, bitemeyenler
```

5.3 Kaynak isteği algoritması

Bir istek gerçekleşmeden önce üç adımda değerlendirilir: (1) istek Need'i aşıyorsa hata; (2) boşta yeterli kaynak yoksa WAIT; (3) istek verilmiş gibi yapılp oluşan durum güvenli mi diye bakılır: SAFE ya da UNSAFE. Değerlendirme gerçek durumu değiştirmez, kopyalar üzerinde çalışır.

5.4 Risk seviyeleri

Risk seviyesi, tetiklenen kuralların en yükseğidir; her kural kendi nedenini ekler. Böylece arayüz yalnızca "HIGH" demek yerine "neden HIGH" sorusunu da cevaplar.

Seviye	Koşul	Örnek neden mesajı
CRITICAL	Tespit algoritması deadlock buldu	DEADLOCK: P1, P2 takılı. Döngü: P1 → R2 → P2 → R1 → P1
HIGH	Banker's güvenli sıra bulamıyor	Sistem güvensiz: Banker's güvenli sıra bulamıyor.
MEDIUM	Bekleyen process var, boş kaynak %20'nin altında ya da döngü var ama deadlock yok	P3 bekliyor: R3 (2 birim) şu an P1 elinde.
LOW	Hiçbiri	Sistem normal çalışıyor.

5.5 Kurtarma

Deadlock'taki her process için bir "sonlandır" seçeneği, elindeki ve birinin beklediği her kaynak için bir "geri al" seçeneği üretilir ve maliyete göre sıralanır. Otomatik kurtarma en ucuz seçeneği uygular, tespiti tekrar çalıştırır ve deadlock bitene kadar sürer. Eşit maliyette kaynak geri alma önce gelir, çünkü process yaşamaya devam eder.

```
def option_cost(process, freed_units):  
    return process.priority * 10 - freed_units * 2 + process.victim_count * 5
```

5.6 Motor ve geri alma

Motor, sistemin her durumunu "başlangıç senaryosu + uygulanan eylemler listesi" olarak tanımlar (event sourcing). Simülasyon deterministik olduğundan "geri al", baştan başlayıp son eylem hariç hepsini yeniden oynatmaktır; bir process sonlandırmasını geri almak bile ek kod gerektirmedi. Her eylem, durum değiştirilmeden **önce** doğrulanır; hatalı bir eylem sistemi yarım bırakmaz.

5.7 Arayüz

Web arayüzü tek sayfalık bir dashboard'dur: senaryo seçimi, adım ilerletme / geri alma / sıfırlama düğmeleri, risk seyri, özet kartları, durum paneli, kurtarma seçenekleri (her birinin yanında "Uygula" düğmesi), senaryo dışı elle eylem formu, RAG çizimi, process ve kaynak kartları ve olay akışı. Her düğme bir POST isteği gönderir ve sayfaya geri yönlendirilir; böylece sayfa yenilendiğinde aynı eylem iki kez uygulanmaz. Sunucu yalnızca yerel adreste (127.0.0.1) çalışır ve yalnızca senaryo klasöründeki dosyaları yükler.

6. Test ve Sonuçlar

6.1 Final demo senaryosu

Final demo senaryosu (*scenarios/final_demo.json*) üç process ve üç kaynakla risk seviyesinin adım adım yükselişini, deadlock'u ve kurtarmayı gösterir:

Adım	Olay	Risk	Banker's
0	Başlangıç	LOW	Güvenli
1	P1, R1'den 1 birim istiyor	LOW	SAFE
2	P1, R3'ten 1 birim istiyor	LOW	SAFE
3	P3, R3'ten 2 birim istiyor (boşta 1 var, bekler)	MEDIUM	WAIT
4	P2, R2'den 1 birim istiyor	HIGH	UNSAFE: uyarı
5	P1, R2'den 1 birim istiyor (bekler)	HIGH	WAIT
6	P2, R1'den 1 birim istiyor: P1 ile P2 birbirini bekliyor	CRITICAL	WAIT
7	Recovery: Sonlandır P1 (maliyet 6, önerilen)	MEDIUM	Güvenli (P2 → P3)
8–10	P2 ve P3 işini bitirip kaynakları bırakıyor	LOW	Güvenli

Banker's Algorithm sistemin güvensiz duruma düştüğünü 4. adımda, yani deadlock oluşmadan **iki adım önce** bildirmektedir. Deadlock anında en ucuz seçenek P1'i sonlandırmaktır: P1 hem R1'i hem R3'ü tuttuğu için sonlandırılması 2 birim serbest bırakır.

Deadlock Guardian

final_demo.json

Senaryoyu yükle

FINAL DEMO: 3 process, 3 kaynak. Normal çalışma (LOW) → P3 beklemeye geçer (MEDIUM) → P2'nin isteği sistemi güvensiz yapar (HIGH) → P1 ile P2 birbirini bekler, deadlock (CRITICAL) → önerilen kurban P1 sonlandırılır → sistem SAFE durumuna döner ve processler işini bitirir.

Sıfırla

Geri al

Sonraki olay

Olay 4 / 10 · Sıradaki: P1, R2'den 1 birim istiyor

Risk seyri: 0 1 2 3 4

Aktif process
3

Kaynak (boş / toplam birim)
1 / 4

Deadlock
yok

Risk
HIGH

Durum **RISK: HIGH**

- Sistem güvensiz: Banker's güvenli sıra bulamıyor.
- P3 bekliyor: R3 (2 birim) şu an P1 elinde.

⚠ **Son istek sistemi güvensiz duruma soktu (Banker's bu isteği onaylamazdı).**

🛡 **GÜVENSİZ** durum: güvenli sıra yok, deadlock oluşabilir.

Son eylem: 4. P2, R2'den 1 birim istiyor

Eylem

P1

R1

1

İste

Bırak

Senaryo dışı bir eylem; "Geri al" ile geri alınabilir.

Resource Allocation Graph

Processler ve kaynaklar

P1
Durum: **READY**
Elindeki kaynaklar: R1x1, R3x1
Max: R1x1, R2x1, R3x1 · Need: R2x1

P2
Durum: **READY**
Elindeki kaynaklar: R2x1
Max: R1x1, R2x1 · Need: R1x1

P3
Durum: **WAITING**
Elindeki kaynaklar: —
Max: R3x2 · Need: R3x2

R1 (0/1 boşta)
Dağıtım: P1x1
Bekleyen kuyruk: —

R2 (0/1 boşta)
Dağıtım: P2x1
Bekleyen kuyruk: —

R3 (1/2 boşta)
Dağıtım: P1x1
Bekleyen kuyruk: P3(ister 2)

Olay akışı

#	Tür	Process	Kaynak	Birim	Ayrıntı
4	ACQUIRED	P2	R2	1	
3	WAITING	P3	R3	2	boşta yalnızca 1 var

Şekil 2 — 4. adım (HIGH): P2'nin isteği sistemi güvensiz duruma soktu; henüz deadlock yok.

Deadlock Guardian — Proje Raporu

Sayfa 10

Deadlock Guardian

final_demo.json
Senaryoyu yükle

FINAL DEMO: 3 process, 3 kaynak. Normal çalışma (LOW) → P3 beklemeye geçer (MEDIUM) → P2'nin isteği sistemi güvensiz yapar (HIGH) → P1 ile P2 birbirini bekler, deadlock (CRITICAL) → önerilen kurban P1 sonlandırılır → sistem SAFE durumuna döner ve processler işini bitirir.

Sıfırla
Geri al
Sonraki olay

Olay 6 / 10 · Sıradaki: Recovery (otomatik, en ucuz seçenek)

Risk seyri: 0 1 2 3 4 5 6

Aktif process
3

Kaynak (boş / toplam birim)
1 / 4

Deadlock
VAR

Risk
CRITICAL

Durum RISK: CRITICAL

- DEADLOCK: P1, P2 takılı. Döngü: P1 → R2 → P2 → R1 → P1
- Sistem güvensiz: Banker's güvenli sıra bulamıyor.
- P2 bekliyor: R1 (1 birim) şu an P1 elinde.
- P1 bekliyor: R2 (1 birim) şu an P2 elinde.
- P3 bekliyor: R3 (2 birim) şu an P1 elinde.

GÜVENSİZ durum: güvenli sıra yok, deadlock oluşabilir.

Son eylem: 6. P2, R1'den 1 birim istiyor

Kurtarma seçenekleri

maliyet = öncelik×10 – serbest bırakılan birim×2 + kurban sayısı×5

Seçenek	Maliyet	
Sonlandır: P1	6	☒ Uygula (önerilen)
Geri al: R1, sahibi P1	8	Uygula
Geri al: R3, sahibi P1	8	Uygula
Geri al: R2, sahibi P2	8	Uygula
Sonlandır: P2	8	Uygula

Elle eylem

P1
R1
1
İste
Bırak

Senaryo dışı bir eylem; "Geri al" ile geri alınabilir.

Resource Allocation Graph

Processler ve kaynaklar

P1 DEADLOCK
Durum: WAITING
Elindeki kaynaklar: R1×1, R3×1
Max: R1×1, R2×1, R3×1 · Need: R2×1

P2 DEADLOCK
Durum: WAITING
Elindeki kaynaklar: R2×1
Max: R1×1, R2×1 · Need: R1×1

P3
Durum: WAITING
Elindeki kaynaklar: —
Max: R3×2 · Need: R3×2

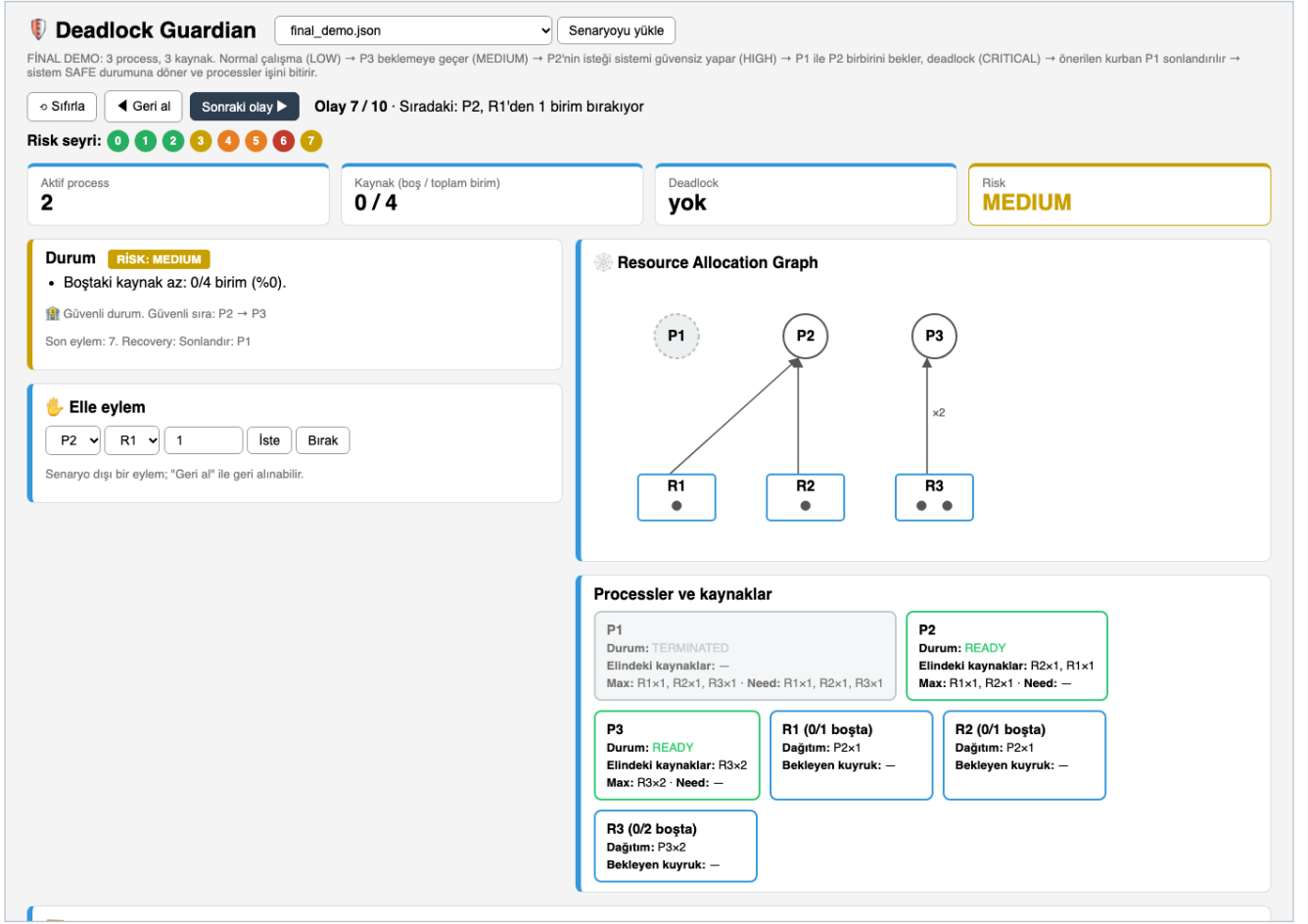
R1 (0/1 boşta)
Dağıtım: P1×1
Bekleyen kuyruk: P2(ister 1)

R2 (0/1 boşta)
Dağıtım: P2×1
Bekleyen kuyruk: P1(ister 1)

R3 (1/2 boşta)
Dağıtım: P1×1
Bekleyen kuyruk: P3(ister 2)

Olay akışı

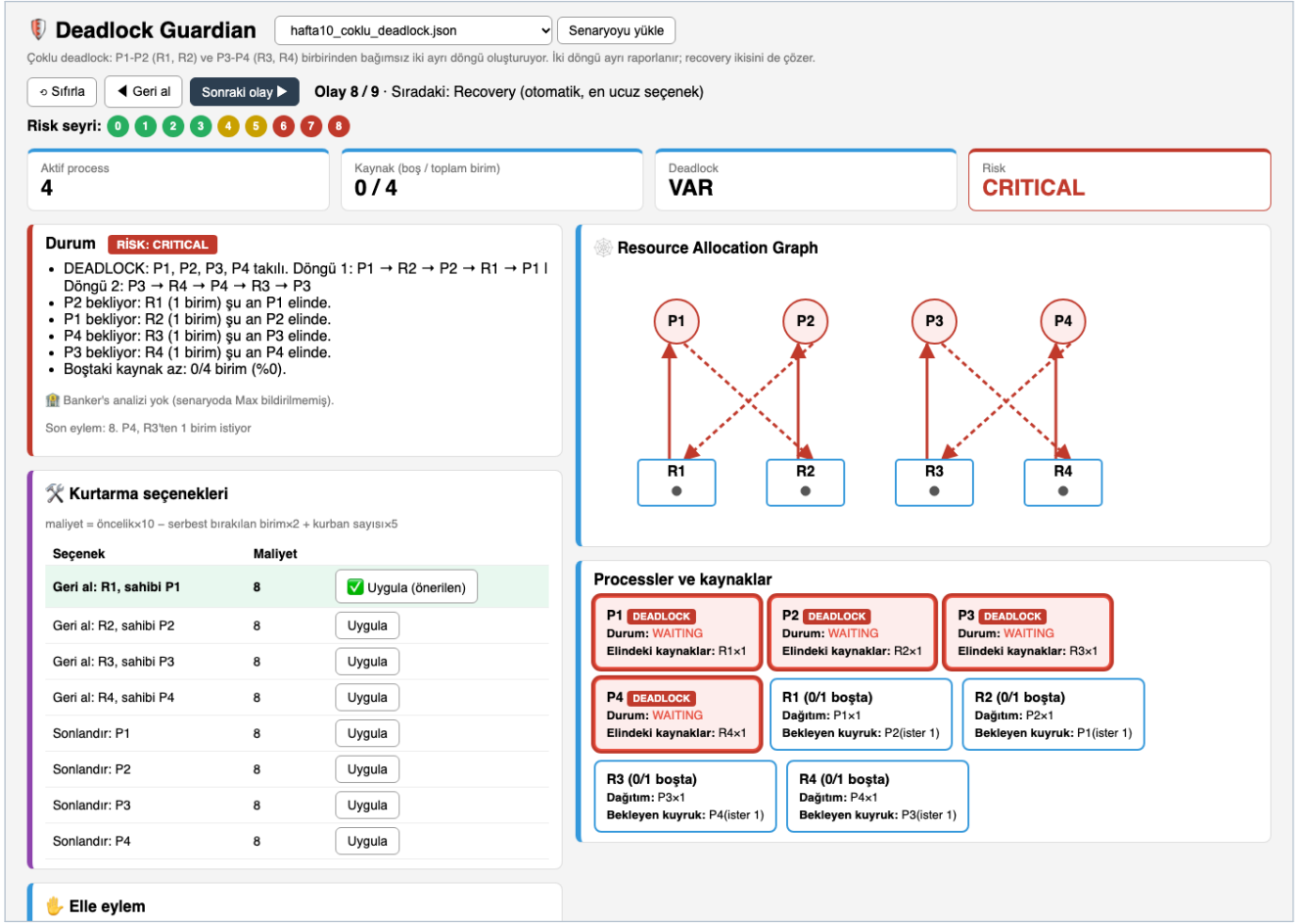
Şekil 3 — 6. adım (CRITICAL): P1 → R2 → P2 → R1 → P1 döngüsü kırmızıyla vurgulanıyor; kurtarma seçenekleri maliyetleriyle listeleniyor.



Şekil 4 — 7. adım: P1 sonlandırıldı (gri); R1 P2'ye, R3'ün iki birimi P3'e geçti; Banker's yeniden güvenli sıra buluyor.

6.2 Sistem testleri

No	Senaryo	Beklenen sonuç	Gerçek sonuç
1	Normal çalışma	Deadlock yok, risk LOW	9 adımın hepsi LOW, her istek SAFE
2	Tek deadlock (P1-R1-P2-R2)	P1 ve P2 deadlock'ta, CRITICAL	Beklendiği gibi
3	Çoklu deadlock (iki ayrı döngü)	İki döngü ayrı raporlanır	İki döngü raporlandı; kurtarma ikisini iki adımda çözdü
4	Deadlock + kurtarma	Deadlock kalmaz, sistem SAFE	Beklendiği gibi
5	Döngü var ama deadlock yok	Deadlock yok	Risk en fazla MEDIUM, hiç CRITICAL değil



Şekil 5 — Çoklu deadlock: iki bağımsız döngü ayrı ayrı raporlanıyor ve grafta vurgulanıyor.

6.3 Otomatik testler ve kod kapsamı

Her hafta yazılan kod pytest ile test edildi; her hafta bütün önceki testler yeniden çalıştırıldı (regresyon).

Toplam **140 test** var ve hepsi geçiyor; satır kapsamı **%99**'dur. Kapsam dışı kalan 4 satır, programın yalnızca komut satırından doğrudan çalıştırıldığında devreye giren satırlar ve bir hata ayıklama metnidir.

Test türü	Örnek
Birim	DFS döngü arama, güvenlik algoritması, maliyet formülü, Türkçe ek seçimi
Entegrasyon	Senaryo + Banker's; motor + tespit + kurtarma + geri al
Sistem	Senaryo dosyaları baştan sona; web arayüzünün düğmeleri (Flask test istemcisi)
Kitapla doğrulama	Silberschatz 8.6.3.3: güvenli sıra <P1, P3, P4, P0, P2>; P1'in isteği SAFE, ardından P4'ünki WAIT, P0'ünki UNSAFE

6.4 Testlerle bulunan hatalar

Uç durumlar sistematik olarak denendi; beş hata bulundu ve her biri için önce hatayı gösteren bir test yazıldı, sonra hata düzeltildi:

Durum	Hata	Düzeltilme
Toplam adetten fazla istek	Process sonsuza kadar bekliyor, tespit algoritması görmüyordu	İstek reddediliyor
Negatif / sıfır miktar	Kaynak birimi "üretiliyor" ya da boş kayıt oluşuyordu	Miktar en az 1 olmalı
Bekleyen process'in eylemi	Process iki kuyrukta birden duruyor, durumu yanlış görünüyordu	Bekleyen process bloke durumda; eylem reddediliyor
Senaryoda olmayan kaynak	Web sayfası çöküyordu	Anlaşılır hata mesajı
İki bağımsız deadlock	Raporda yalnızca biri görünüyordu	Bütün ayırık döngüler raporlanıyor

7. Sonuç ve Gelecek Çalışmalar

7.1 Başarı kriterleri

Kriter	Durum
En az bir deadlock senaryosunu doğru tespit etmek	Sağlandı: tek, çoklu ve çok örnekli kaynaklı senaryolar
Deadlock'a dahil process ve kaynakları göstermek	Sağlandı: durum paneli, kart vurgusu, graf üzerinde döngü
Belirlenen koşullarda deadlock risk uyarısı vermek	Sağlandı: dört risk seviyesi ve nedenleri, Banker's uyarısı
En az bir recovery yöntemi uygulamak	Sağlandı: sonlandırma ve kaynak geri alma
Recovery sonrası sistemin güvenli duruma döndüğünü göstermek	Sağlandı: Banker's güvenli sıra buluyor
Normal ve deadlock durumlarını karşılaştırmak	Sağlandı: normal çalışma senaryosu ve risk seyri
Kodun modüler olması	Sağlandı: katmanlı mimari; algoritmalar arayüzden bağımsız test edildi
Baştan sona çalışan bir demo	Sağlandı: final_demo.json ve web arayüzü

7.2 Sınırlamalar

- Sistem bir **simülasyondur**; gerçek işletim sistemi process'lerini ya da kilitlerini izlemez.
- Risk seviyeleri bir **sezgisel kuraldır** (heuristic); %20 eşiği ekip kararıdır, kesin garanti vermez.
- Banker's güvensiz isteği yalnızca uyarır, engellemez; Max bildirilmeyen senaryolarda HIGH seviyesi değerlendirilemez.
- Kaynak geri almada rollback basitleştirilmiştir: process yalnızca o kaynağı almadan önceki noktaya döner.
- Bekleme kuyruğu karşılanabilen istekleri öne alarak servis eder; büyük istekler geride kalabilir (starvation).
- Web arayüzü tek kullanıcılıdır ve yerel demo için tasarlanmıştır.

7.3 Gelecek çalışmalar

- **Kaçınma modu**: Banker's'ın güvensiz isteği engellediği, ayarlanabilir bir mod.
- **Gerçek sisteme bağlantı**: Bir programdaki kilit (mutex) kullanımını izleyip aynı RAG'e dönüştürmek; örneğin veritabanlarındaki kilit bekleme grafları (wait-for graph) bu yaklaşımla çalışır.
- **Önleme stratejileri**: Kaynaklara sıra numarası vererek döngüsel beklemeyi imkânsız kılan kaynak sıralama (resource ordering) yönteminin karşılaştırmalı gösterimi.
- **Zamanlama**: Olayların senaryodan değil, zamanlayıcıyla rastgele üretildiği bir yük testi modu.

8. Kaynakça

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 10. baskı, Wiley, 2018. Bölüm 8: Deadlocks.
- E. G. Coffman, M. J. Elphick, A. Shoshani, "System Deadlocks", *ACM Computing Surveys*, 3(2), s. 67–78, 1971.
- R. C. Holt, "Some Deadlock Properties of Computer Systems", *ACM Computing Surveys*, 4(3), s. 179–196, 1972.
- E. W. Dijkstra, "Cooperating Sequential Processes", EWD123, Technische Hogeschool Eindhoven, 1965.
- Flask belgeleri: flask.palletsprojects.com
- pytest ve pytest-cov belgeleri: docs.pytest.org

9. Ekip Katkıları

Modüller, proje planında belirlenen dört role göre ayrılmıştır. Sorumlu sütunu ekip tarafından doldurulacaktır.

Roİ	Modüller	Sorumlu
Simülasyon ve veri modeli	process.py, resource.py, event_log.py, engine.py, scenario_loader.py, scenarios/	
Tespit ve kurtarma	detection.py, recovery.py	
Risk analizi	bankers.py, risk.py	
Arayüz ve görselleştirme	app.py, dashboard_view.py, graph_view.py, html_view.py, run_scenario.py	
Test ve belgeler	tests/, Test Raporu, bu rapor	